Reflection Naming: fix reflexpr

Document #:	P2087R0
Date:	2020-01-12
Project:	Programming Language C++ SG7

Reply-to:	Mihail Naydenov <mihailnajdenov@gmail.com>

§ Abstract

This paper suggests an alternative keyword for `reflexpr`, part of the current Reflection API[1], with the aim to increase consistency and better match user expectations.

§ Issue

Initiating reflection is currently done via the `reflexpr` keyword. It could be argued, this keyword is both inconsistent with previous conventions and somewhat misleading.

Inconsistent

We already have multiple keywords, all of which do some sort of reflection:

- `sizeof` - returns size of instances of a type.
- `alignof` - returns required alignment for instances of a type.
- typeof - returns the type of an expression. If this wasn't already an extension, would have been used instead of `decltype`.
- `offsetof` - returns the offset from the beginning of an object.
- `std::addressof` - returns the address of an object.

As you can see, the functions are consistent b/w each other, following the what-we-need + "of" naming scheme. There is no reason to not respect this model with the Reflection API as well.

Misleading

`reflexpr` "borrows" the "expr" ("expression") suffix from `constexpr`, *but uses it with a different meaning:*

- `constexpr` means "Define an expression of type constant [evaluated]".

— `reflexpr` means “Reflect this [expression?]”

In the former case “expr” is used to *define new type of expression*.

In the latter case “expr” does *not* denote a new type of expression. It actually *does not denote anything*, beyond “this is a reflection [expression]”, as the argument might not be a C++ expression, strictly speaking, and the end result is just a normal (consteval) expression, not a special, “reflection” one.

Using “expr” not only creates misleading “symmetry”, one that is more confusing then helpful (it is not helpful because it does not *hint* at anything), but also prevents us from introducing this syntax to *actually mean “reflection expression”*. For example, we could image `reflexpr T == int` (“is same”), `reflexpr A : T` (“is base of”), `reflexpr A(T) || T(B)` (“constructable from”), etc. In other words, we *might* want to use `reflexpr` to do *inline reflection*, without going through full reflection (meta::info).

> At the moment Reflection envisions mirroring type_traits into the its own namespace. This will not remove the need for type_traits, because going back and forth b/w reflection and program code is cumbersome and verbose, probably unavoidably so.

§ Proposed solution

Do not use the “expr” suffix and use “of” instead.
This way we fix *both* issues, stated above - we break the misleading connection to `constxpr` *and* we increase consistency with the current facilities.

Suggested names are: `reflof` (proposed), or alternatively `reflectof` or `reflectionof`.

> Another alternative is to use “on”, simply because it matches the verb “to reflect” (*on* something). `reflecton`, `reflon`.

---

1. Reflection: http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1240r1.pdf↩